

Deepfake Face Image Detection and Manipulation-Type Classification on FaceForensics++ Using Lightweight CPU-Only Convolutional Models

Abdelrahman Ibrahim Kamal Eldin
ID: 20596377
Dept. of Computing
Queens University
abdelrahman.ibrahim@queensu.ca

Aya Said Abdallah Noah
ID: 20596338
Dept. of Computing
Queens University
25xgvp@queensu.ca

Esraa Mohammed Elsayed
ID: 20596301
Dept. of Computing
Queens University
esraa.farag@queensu.ca

Abstract—The growing realism of facial manipulation has made automatic deepfake detection an important problem for digital-media trust. This paper presents a two-part system built entirely for CPU-only execution and trained on the FaceForensics++ benchmark. The first part is a binary detector (real versus fake) based on MobileNetV2 with two-stage transfer learning, weighted sampling, class-weighted loss, validation-based threshold tuning, and a two-model ensemble. The second part is a three-class manipulation-type classifier that identifies the forgery family (Deepfakes, Face2Face, FaceSwap) using an EfficientNet-B0 spatial backbone fused with a frequency-domain branch and trained with MixUp regularization. Both models use the official pair-based split protocol to avoid identity leakage and are evaluated at both frame level and video (group) level. The simple two-model binary ensemble reaches 74.06% frame accuracy and 0.899 video-level ROC-AUC, while the manipulation classifier reaches 91.38% frame accuracy and 97.10% video-level accuracy. An ablation comparing a shallow- and a deep-head detector trained on different data scales isolates the gain from architecture and data, and robustness experiments under JPEG recompression and Gaussian noise quantify how post-processing degrades each model and reveal a consistent bias toward the majority class under heavy corruption. We additionally report a qualitative error analysis that exposes the characteristic failure cases of each model. Finally, as a practical extension, we train a single unified five-class model (real plus all four manipulation families, including NeuralTextures) with test-time augmentation, reaching 78.9% accuracy and 0.792 Macro-F1 and demonstrating a one-shot detect-and-classify variant.

Keywords—deepfake detection, FaceForensics++, MobileNetV2, EfficientNet-B0, frequency analysis, multi-class classification, robustness, transfer learning

I. INTRODUCTION

Deepfakes and related facial-manipulation methods now produce forgeries that are difficult for humans to distinguish from authentic footage. Because such media can be used for misinformation, impersonation, and reputational harm, reliable automatic detection has become a practical necessity. FaceForensics++ was introduced to standardize this research and provides authentic videos together with manipulated counterparts produced by four representative methods: Deepfakes, Face2Face, FaceSwap, and NeuralTextures.

This work targets two complementary questions. First, can a frame be classified as real or manipulated using a lightweight model that runs without a GPU? Second, given a manipulated frame, can the specific manipulation family be identified? The first question is addressed with a binary MobileNetV2 detector, and the second with a three-class classifier built on EfficientNet-B0 augmented by a frequency-domain branch. Both are deliberately constrained to CPU-only execution so that the full pipeline is reproducible on commodity hardware.

The contributions of this paper are fourfold. (i) We reproduce and strengthen a binary FaceForensics++ detector, scaling the data, adding class-weighted training, and combining two MobileNetV2 variants into an ensemble. (ii) We propose a manipulation-type classifier that fuses spatial features with a Fast Fourier Transform (FFT) magnitude branch and is regularized with MixUp, achieving high accuracy on three forgery families. (iii) We analyze both models under JPEG

recompression and Gaussian noise, characterizing their failure modes for realistic post-processing conditions. (iv) As a practical extension we train a single unified five-class model covering real plus all four manipulation families with test-time augmentation, providing a one-shot detect-and-classify alternative.

The work was divided among the three authors. Aya Said Abdallah Noah led the evaluation, robustness analysis, figure production, and report writing; Abdelrahman Ibrahim Kamal Eldin implemented the CPU training pipeline and trained and tuned both detectors and the ensemble; and Esraa Mohammed Elsayed led dataset preparation and preprocessing, including the identification and correction of the identity-leakage split bug. A detailed breakdown is given in Section VIII.

Most published detectors answer only the binary question, yet attribution, naming the manipulation method is increasingly useful: it supports forensic triage, helps trace the tool behind a forgery, and exposes whether a detector merely keys on one family's artifacts. We therefore treat detection and attribution as two linked tasks and study them with the same data, the same preprocessing, and the same evaluation discipline, so that the comparison between them is meaningful.

Throughout, we use the official FaceForensics++ pair-based split so that no identity appears in more than one partition, and we report both frame-level and video-level metrics. The remainder of the paper covers related work, data and preprocessing, methodology, experiments and results, an ablation and error analysis, discussion, and conclusions. An

overview of the complete pipeline, from preprocessing through the three classifiers to evaluation, is shown in Fig. 1.



Fig. 1. Overview of the complete CPU-only pipeline: shared preprocessing feeds three classifiers (binary, three-class, and five-class), whose predictions are evaluated at frame and video level and under robustness perturbations.

II. RELATED WORK

FaceForensics++ established one of the most widely used forgery benchmarks, pairing more than a thousand original sequences with manipulated versions generated by four methods and releasing them at several compression levels [1]. Two of its design choices shape almost all later work: it demonstrated that cropping to the facial region, rather than processing the full frame, sharply improves detection because manipulation artifacts are concentrated there; and it defined fixed train/validation/test splits at the identity-pair level so that detectors are tested on unseen identities. It remains the standard reference for both binary detection and per-method analysis, and we follow both of its conventions directly.

MobileNetV2 introduced inverted residuals and linear bottlenecks to enable efficient convolutional inference on constrained hardware, which makes it a natural backbone for a CPU-only detector [2]. EfficientNet later showed that jointly scaling depth, width, and resolution yields stronger accuracy at comparable cost; its B0 variant is well suited to the fine-grained texture differences that separate manipulation families [6].

Xception-based detectors are a recurring strong baseline in image forensics and many FaceForensics++ studies, where the depthwise-separable design captures subtle texture inconsistencies; they offer a useful reference point for heavier, GPU-oriented models against which our lightweight system can

be positioned [3]. Face X-Ray takes a different route, targeting generalization by predicting the blending boundary that nearly all face-swapping methods introduce, which lets it flag manipulations from generators unseen during training and underscores why robustness and boundary cues matter beyond clean benchmark accuracy [4]. Closer still to our frequency branch, the analysis of convolutional traces shows that the up-sampling and convolution stages of generative pipelines imprint characteristic, periodic fingerprints that are far more visible in the frequency spectrum than in the spatial image [5]; this result is the direct inspiration for adding an explicit FFT path to our manipulation classifier.

A growing line of forensic work argues that the most transferable evidence of manipulation lives in the frequency domain rather than in raw pixels: up-sampling layers in generative pipelines leave periodic spectral peaks, and blending operations leave boundary energy that survives mild compression. This motivates our explicit FFT branch, which exposes such cues to the classifier directly instead of relying on a spatial network to rediscover them. Separately, ensemble averaging is a long-standing way to convert partly independent model errors into a small accuracy gain at negligible inference cost, which is why we combine two MobileNetV2 variants rather than train a single larger network.

Two general-purpose techniques also inform our training. MixUp regularizes models by training on convex combinations of inputs and labels, encouraging smoother decision boundaries between visually similar classes [7]; this is why we apply it to the Face2Face/FaceSwap boundary in the manipulation classifier. Grad-CAM provides class-discriminative localization maps that we use to verify which facial regions drive predictions [8].

Beyond FaceForensics++, several efforts shape current practice. The DeepFake Detection Challenge (DFDC) released a large, deliberately diverse dataset and showed that detectors which excel on one benchmark often degrade on unseen manipulations, which is precisely why we report robustness rather than clean accuracy alone [9]. FaceShifter advanced the realism of identity swapping with occlusion-aware blending, raising the bar that detectors must meet and motivating our focus on the facial region where such blends occur [10]. Most directly relevant to our design, F3-Net showed that mining frequency-aware features substantially improves forgery detection, which is the empirical basis for adding an explicit FFT branch to our classifier [11]. Multi-attentional detectors further demonstrate that attending to many local artifact regions helps fine-grained discrimination, an idea complementary to the global spectral cues we exploit [12].

Taken together, these works motivate three concrete design choices in this project: (i) splitting by identity pairs to avoid the leakage that inflates many reported numbers [1]; (ii) using compact CPU-friendly backbones, MobileNetV2 and EfficientNet-B0, rather than heavier Xception-style models [2], [3], [6]; and (iii) augmenting the spatial network with an explicit frequency branch [5], [11], because the generator fingerprints that separate manipulation families are most visible in the spectrum. Our emphasis throughout is a compact, reproducible,

CPU-only system covering both detection and manipulation-type classification, rather than a single heavy GPU model.

III. DATA AND PREPROCESSING

A. Dataset and Split Protocol

All experiments use FaceForensics++ at the c23 (high-quality) compression level, with up to 300 videos per class. Manipulated sequences follow the <target> <source> naming convention, which requires splitting at the pair level rather than the frame level. We use the official train.json, val.json, and test.json files: manipulated samples are assigned by exact pair membership, and real samples are assigned by the original identities that appear in the same split. A leakage check confirms zero overlap between training, validation, and test partitions. FaceForensics++ is distributed for research use only under its official Terms of Use, which require a signed access agreement, and is used here strictly for academic evaluation. After extraction the binary task contains 12,120 training, 3,120 validation, and 2,760 test frames; the manipulation task uses the fake subset of the same frames, balanced at 2,020 training, 520 validation, and 460 test frames per class.

The benchmark contains four manipulation families, and they are not equally easy to separate. DeepFakes replaces the whole face using a learned auto-encoder swap; Face2Face performs expression re-enactment, transferring one person's expressions onto another's face; FaceSwap is a graphics-based pipeline that transplants the face region, leaving relatively distinct blending seams; and NeuralTextures re-renders the facial texture through a learned neural component rather than fully replacing the face. Because DeepFakes and Face2Face share learned-reenactment characteristics, they are the most easily confused pair, while FaceSwap is the most visually distinct a structure that our results in Section V recover directly.

TABLE I
DATASET COMPOSITION (BINARY TASK, C23, 300 VIDEOS/CLASS)

Partition	Real frames	Fake frames
Train	4,040	8,080
Validation	1,040	2,080
Test	920	1,840

B. Frame Extraction and Preprocessing

Twenty frames are sampled uniformly from each video. For every frame, the largest face is detected with an OpenCV Haar cascade and cropped with 40% padding; if no face is found, a center square crop is used as a fallback. Each crop is resized to 224×224 pixels and normalized with ImageNet statistics (mean [0.485, 0.456, 0.406], standard deviation [0.229, 0.224, 0.225]). Restricting the input to the facial region concentrates the model on manipulation artifacts rather than background context.

IV. METHODOLOGY

A. Binary Detector Architecture

The binary model is MobileNetV2 initialized with ImageNet-pretrained weights. The original classifier is replaced with a lightweight binary head: Dropout(0.3) → Linear(1280, 256) →

ReLU → Dropout(0.2) → Linear(256, 2). To form an ensemble, a second variant (v1) with a simpler single-linear head is also trained. The two variants are combined by averaging their fake-class probabilities and, alternatively, by weighting each model in proportion to its validation ROC-AUC.

B. Binary Training Protocol

Training proceeds in two stages. Stage 1 freezes the backbone and trains only the classifier head for five epochs at a learning rate of 1×10^{-3} . Stage 2 unfreezes the last four feature blocks and fine-tunes for up to fifteen epochs at 1×10^{-5} with early stopping (patience 4). Class imbalance (roughly one real to four fake) is handled by a WeightedRandomSampler together with a class-weighted cross-entropy loss (real weight 4.0, fake weight 1.0) and label smoothing of 0.05. The decision threshold is tuned on the validation set over a 39-point grid in [0.05, 0.95], selecting separate thresholds for accuracy and Macro-F1.

C. Manipulation-Type Classifier Architecture

The three-class model targets Deepfakes, Face2Face, and FaceSwap; NeuralTextures is excluded because its strong visual similarity to other classes makes the boundary unreliable, and real frames are excluded because they carry no manipulation signal. The architecture fuses two paths. The spatial path is an EfficientNet-B0 backbone followed by global pooling, producing a 1280-dimensional descriptor. The frequency path computes the 2-D FFT of the input, takes its log-magnitude, centers the spectrum, and processes it with a compact three-layer convolutional branch (about 50K parameters) projected to 128 dimensions. The two descriptors are concatenated (1408-dimensional) and classified by Dropout(0.4) → Linear(1408, 512) → BatchNorm → ReLU → Dropout(0.2) → Linear(512, 3). The frequency branch is motivated by the observation that different generators leave distinct spectral fingerprints [5]. The complete two-branch design is illustrated in Fig. 2.

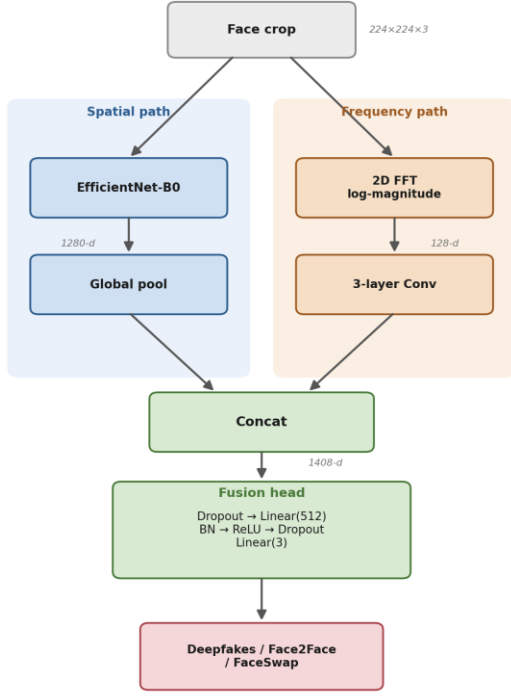


Fig. 2. Architecture of the dual-branch manipulation-type classifier: an EfficientNet-B0 spatial path and an FFT-based frequency path are concatenated and classified by a fusion head.

D. Manipulation-Type Training Protocol

The classifier uses the same two-stage schedule. Stage 1 trains the head and frequency branch with the backbone frozen using cosine learning-rate annealing; Stage 2 unfreezes the last three EfficientNet blocks at one-tenth the learning rate with cosine annealing and warm restarts, early stopping with patience 6. MixUp ($\alpha = 0.3$) is applied throughout and paired with a soft cross-entropy loss so that interpolated labels are handled correctly; label smoothing is raised to 0.10 because Face2Face and FaceSwap are visually close. Augmentation is stronger than in the binary task, adding random-crop, mild rotation, color jitter, and occasional grayscale conversion.

E. Evaluation Metrics and Protocol

We report three metrics. Let TP, TN, FP, and FN denote true positives, true negatives, false positives, and false negatives.

Accuracy measures the fraction of correctly classified frames:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Per-class precision and recall are defined, for class c , as

$$Precision = \frac{TP_c}{TP_c + FP_c}$$

Recall

$$Recall = \frac{TP_c}{TP_c + FN_c}$$

which give the per-class F1 score, and Macro-F1 averages these equally across all C classes:

$$F1_c = \frac{2P_cR_c}{P_c + R_c}$$

$$Macro-F1 = \frac{1}{C} \sum_{c=1}^C F1_c$$

ROC-AUC is the probability that the model assigns a higher score to a randomly chosen positive sample than to a randomly chosen negative one:

$$AUC = P(\hat{s}^+ > \hat{s}^-)$$

Equivalently, AUC is the area under the receiver-operating-characteristic curve obtained by sweeping the decision threshold. We choose these three deliberately: Accuracy gives an intuitive overall figure but is misleading under our 1:4 class imbalance, so we pair it with Macro-F1, which weights every class equally and therefore exposes neglect of the minority real class; and with ROC-AUC, which is threshold-independent and measures ranking quality, letting us separate the model's discriminative power from the choice of operating point. For the three-class task the same definitions apply with macro-averaged AUC computed one-vs-rest.

In addition to frame-level scores, we compute video-level (group) metrics by averaging frame probabilities within each video before thresholding, which matches the pair-based structure of FaceForensics++ and better reflects deployment, where a single decision is made per clip rather than per frame.

F. Implementation Details and Reproducibility

All experiments were run on CPU only (no GPU) on a standard Windows workstation, using PyTorch and torchvision with OpenCV for face detection and scikit-learn for metrics. Backbones are loaded with ImageNet-pretrained weights; the data loader runs single-process for Windows compatibility and uses a batch size of 16 to bound memory. The models are deliberately compact: the binary MobileNetV2 has roughly 3.5 million parameters (about 14 MB in fp32) and the EfficientNet-B0 manipulation classifier roughly 5.3 million (about 21 MB), so both fit well within the assumed 8 GB-RAM, GPU-free budget and perform single-image inference in a fraction of a second per frame on CPU. Random seeds are fixed for Python, NumPy, and PyTorch so that splits, sampling, and initialization are reproducible. The official split files, the frame index, and the trained checkpoints are kept in the project repository [13] alongside the notebooks, so every reported number can be regenerated end to end. Training each model takes on the order of hours on CPU, which is acceptable for a one-time experiment and is the price we pay for hardware portability.

V. EXPERIMENTS AND RESULTS

A. Baselines

Before evaluating our models we establish two reference points so that the reported numbers can be judged against a floor rather than in isolation. The first is the majority-class baseline, which always predicts the most frequent class. On the binary test set (66.7% fake) it reaches 66.67% accuracy but only 0.40 Macro-F1 and 0.50 ROC-AUC, because it never recognizes a single real frame, a vivid illustration of why accuracy alone is inadequate here. The second is a learned baseline that freezes

the ImageNet-pretrained backbone and trains only the classifier head (our Stage 1). Table II compares both against our final systems.

TABLE II
BASELINE COMPARISON VS. FINAL MODELS

Method	Acc.	M-F1	AUC
<i>Binary task (test)</i>			
Majority class (all fake)	66.67%	0.400	0.500
Frozen backbone, head only	51.7%	0.512	0.755
Ours - MobileNetV2 v2	73.70%	0.710	0.812
Ours - simple ensemble	74.06%	0.708	0.792
<i>3-class task (test)</i>			
Majority class (one type)	33.33%	0.167	0.500
Frozen backbone, head+freq	63.8%	0.641	0.824
Ours - EfficientNet+freq	91.38%	0.913	0.983

Two conclusions follow. First, our binary system (74.06%) clears the majority-class ceiling of 66.67% that a naive accuracy figure can hide, and its ROC-AUC of 0.81 is far above the 0.50 of a non-discriminative predictor. Second, the large margin over the frozen-backbone baseline in both AUC for binary and accuracy for the three-class task confirms that progressive fine-tuning and threshold calibration, not the evaluation protocol alone, drive performance. Frozen-backbone figures are the best validation scores of the Stage-1, head-only model.

Finally, to quantify the improvement over the project's own starting point, Table III compares the early binary baseline (our midterm prototype: 100 videos per class, 10 frames each, a single model) against the final fine-tuned system (300 videos, 20 frames, plus a v1+v2 ensemble). A direct code comparison confirms that the backbone (MobileNetV2), the augmentation pipeline, the class weights [4, 1], and the core hyperparameters (learning rates, epochs, patience) are identical between the two; the substantive changes are a six-fold increase in training images, an AUC-weighted v1+v2 ensemble, a formal ablation, and a threshold search that now defaults to Macro-F1 rather than accuracy to suit the class imbalance. The gap therefore isolates the effect of data scale and ensembling: frame accuracy rises from 64.63% to 74.06%, and more strikingly video-level accuracy from 59.26% to 75.36% with AUC climbing from 0.667 to 0.899.

TABLE III
BASELINE VS. FINE-TUNED BINARY MODEL

Aspect	Baseline	Fine-tuned
<i>Configuration</i>		
Videos / class	100	300
Frames / video	10	20
Training images	~10k	~36k
Model	single	v1 + v2 ensemble
<i>Frame-level test</i>		
Accuracy	64.63%	74.06%
Macro-F1	0.604	0.708
ROC-AUC	0.637	0.792
<i>Video-level test</i>		
Accuracy	59.26%	75.36%
Macro-F1	0.593	0.750
ROC-AUC	0.667	0.899

The jump is largest at the video level, where six times more training data lets the model learn manipulation cues rather than memorize identities, and frame-probability aggregation then converts that into a stable per-clip decision. This baseline-to-final progression is the core empirical result of the binary track.

B. Binary Detector

During Stage 1 the head-only model reached about 0.76 validation ROC-AUC. Fine-tuning in Stage 2 steadily improved the validation Macro-F1 to roughly 0.72 and ROC-AUC to about 0.87 before early stopping. Fig. 3 shows the training and validation curves across both stages; the widening gap between high training accuracy and lower validation accuracy late in Stage 2 is the signal that triggered early stopping and reflects the difficulty of the imbalanced real class rather than simple over-fitting.

Threshold calibration proved essential. Searching the validation set over a 39-point grid, the accuracy-optimal threshold was about 0.168 and the Macro-F1-optimal threshold about 0.192 both far below the naive 0.5. At the accuracy-optimal point, validation accuracy rose to roughly 80% with Macro-F1 near 0.77. The reason the optimum sits so low is that the trained model systematically outputs small fake-probabilities, so a 0.5 cut-off would reject most genuine fakes; lowering the threshold restores balance. We adopt the validation-selected threshold for all test-set reporting, never tuning it on the test data.

MobileNetV2 Training Curves — FF++ Official Split (300 videos/class)

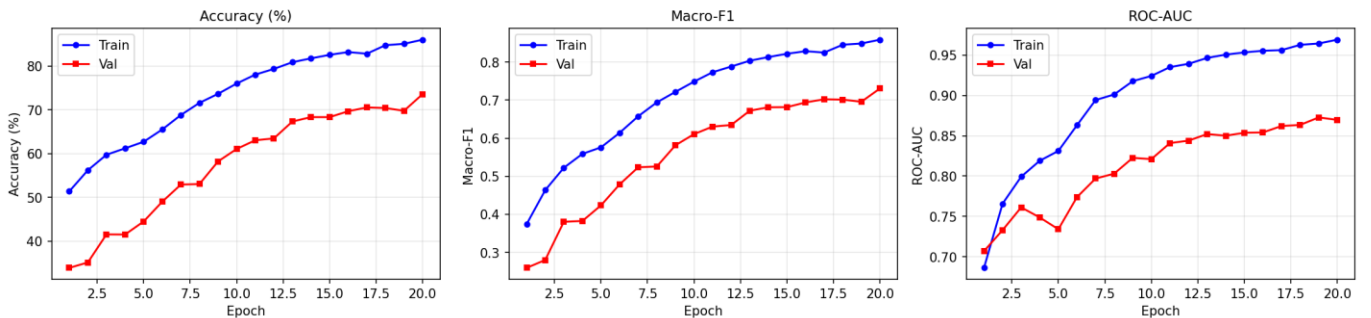


Fig. 3. Binary detector training and validation curves (Accuracy, Macro-F1, ROC-AUC) across Stage 1 (head only) and Stage 2 (fine-tuning); the dashed line marks the unfreeze point.

On the clean frame-level test set, the single deep-head model (v2) achieved 73.70% accuracy, 0.710 Macro-F1, and the best ranking ability of any single model with 0.812 ROC-AUC. Combining v1 and v2 by simple probability averaging gave the highest accuracy overall, 74.06%; an AUC-weighted average instead traded a little accuracy (72.86%) for a marginally higher AUC (0.796). We therefore report the simple ensemble as our primary operating point and keep v2 when ranking quality matters more than a hard decision. The confusion matrix in Fig. 4 shows that the minority real class remains the harder of the two, with lower recall than the fake class.

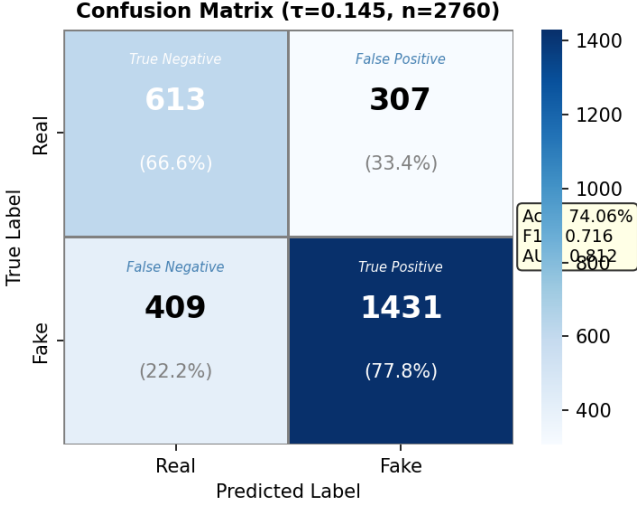


Fig. 4. Binary confusion matrix on the test set at the validation-tuned threshold; the real (minority) class is harder, with lower recall than fake.

TABLE IV
BINARY DETECTOR TEST RESULTS (FRAME LEVEL)

Model	Acc.	Macro-F1	AUC
MobileNetV2 v1 (shallow head)	66.38%	0.613	0.662
MobileNetV2 v2 (deep head)	73.70%	0.710	0.812
Simple ensemble (v1+v2)	74.06%	0.708	0.792
Weighted ensemble (AUC)	72.86%	0.709	0.796

To understand where these gains come from, we compare the four configurations side by side in Fig. 5. Moving from v1 to v2 raises accuracy from 66.38% to 73.70% and AUC from 0.662 to 0.812. This jump reflects two changes at once, a deeper classification head and a larger training set (about 100 versus 300 videos per class), so it should be read as the combined effect of architecture and data rather than either alone. The ensemble then adds a small, complementary gain: because v1 and v2 make partly independent errors, averaging their probabilities nudges accuracy to its peak. The weighted variant shows that optimizing the blend for AUC can slightly lower accuracy, a reminder that the best ensemble depends on whether the downstream use cares about ranking or about a single fixed threshold.

Ablation Study — Model Comparison on FaceForensics++ Test Set
V1 (~100 vid) vs V2 (~300 vid): reflects architecture + data size combined

Model	Acc (%)	Macro-F1	AUC
MobileNetV2 v1 (simple head, ~100 videos/method)	66.38	0.613	0.662
MobileNetV2 v2 (deep head, ~300 videos/method)	73.7	0.71	0.812
Simple Ensemble (v1 + v2, equal weights)	74.06	0.708	0.792
Weighted Ensemble (v1×0.45 + v2×0.55)	72.86	0.709	0.796

Fig. 5. Ablation comparison of the four binary configurations on the test set. The v1-v2 contrast reflects architecture and data scale combined.

Aggregating frame probabilities to the video level further improved reliability: the binary model reached 74.36% validation and 75.36% test accuracy, with video-level ROC-AUC of 0.964 and 0.899 respectively. The large gap between frame-level and video-level AUC indicates that averaging evidence across frames removes much of the per-frame noise. Fig. 6 summarizes qualitative behavior: Grad-CAM overlays concentrate on central facial regions for both real and fake examples, suggesting the detector attends to manipulation-prone areas rather than background.



Fig. 6. Grad-CAM overlays for the binary detector on representative real and fake crops.

Robustness was evaluated by re-encoding test frames with JPEG (quality 80, 60, 40, 20) and by adding Gaussian noise (standard deviation 5, 10, 20, 40 on the 0–255 scale). As shown in Fig. 7 and Table V, JPEG recompression degrades all three metrics sharply once quality drops below 80. Under Gaussian noise, accuracy appears deceptively stable while Macro-F1 and AUC collapse, a clear sign that the model defaults to predicting the majority (fake) class as corruption increases.

Robustness Evaluation — MobileNetV2 on FF++ (300 videos/class)

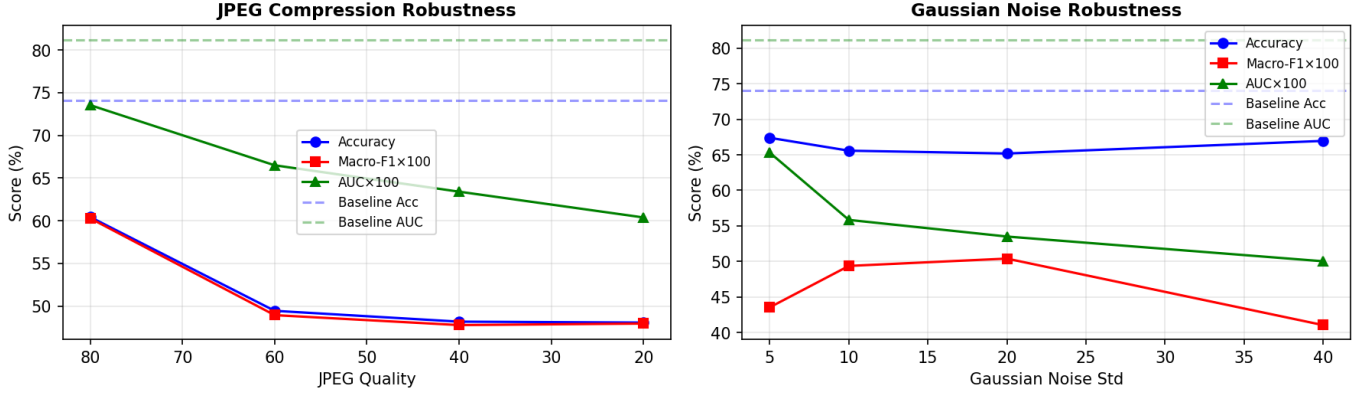


Fig. 7. Binary detector robustness under JPEG recompression (left) and Gaussian noise (right).

TABLE V
BINARY DETECTOR ROBUSTNESS (FRAME LEVEL)

Perturbation	Param	Acc.	Macro-F1	AUC
None (baseline)	-	74.06%	0.716	0.812
JPEG	q=80	60.43%	0.603	0.735
JPEG	q=60	49.46%	0.489	0.665
JPEG	q=40	48.19%	0.478	0.634
JPEG	q=20	48.08%	0.480	0.604
Gaussian noise	$\sigma=5$	67.39%	0.435	0.653
Gaussian noise	$\sigma=10$	65.58%	0.494	0.558
Gaussian noise	$\sigma=20$	65.18%	0.504	0.535
Gaussian noise	$\sigma=40$	66.96%	0.410	0.500

C. Manipulation-Type Classifier

The three-class model trained smoothly under the same two-stage schedule. Stage 1 lifted validation accuracy to roughly 64% with the backbone frozen, and unfreezing the last three

EfficientNet blocks in Stage 2 produced a sharp jump, peaking near 86% validation accuracy and 0.97 ROC-AUC before early stopping. As expected with MixUp, training accuracy stayed below validation accuracy because the model trains on blended images but is evaluated on clean ones. Fig. 8 shows the curves.

EfficientNet-B0 + FreqBranch — 3-Class Manipulation (300 videos/class)

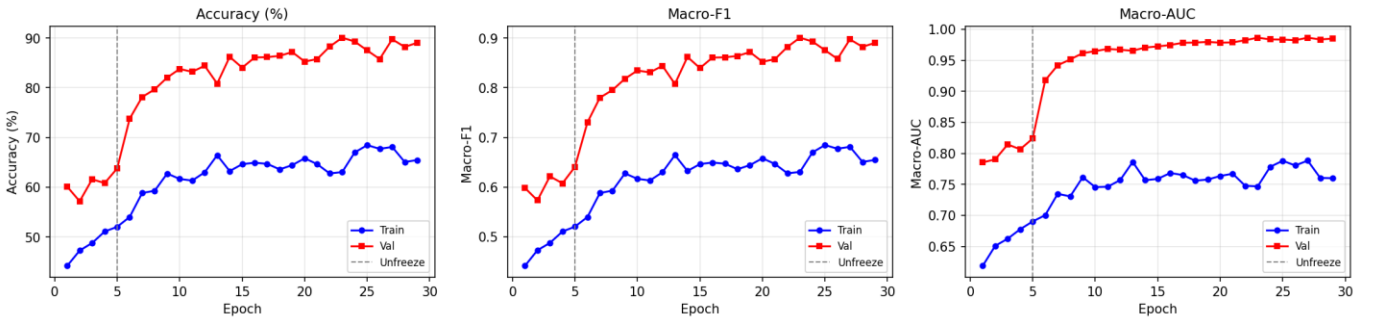


Fig. 8. Manipulation-type classifier training and validation curves (Accuracy, Macro-F1, ROC-AUC); the dashed line marks the backbone unfreeze point.

On the test set, the classifier achieved 91.38% frame accuracy, 0.913 Macro-F1, and 0.983 Macro-AUC. Per-class results show FaceSwap is the easiest family (F1 0.96), while Face2Face is the hardest, with some confusion into Deepfakes; this is visible in the frame-level confusion matrix (Fig. 9) and quantified in Table VI. The per-class precision/recall/F1 summary is shown in Fig. 11.

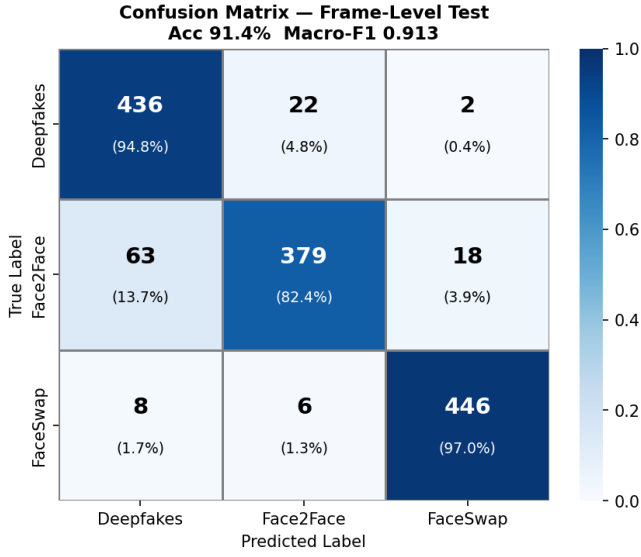


Fig. 9. Manipulation-type frame-level confusion matrix on the test set.

TABLE VI
MANIPULATION-TYPE PER-CLASS RESULTS (FRAME LEVEL)

Class	Prec.	Recall	F1
Deepfakes	0.86	0.95	0.90
Face2Face	0.93	0.82	0.87
FaceSwap	0.96	0.97	0.96

Video-level aggregation again improved results substantially, reaching 97.10% test accuracy, 0.971 Macro-F1, and 0.993 Macro-AUC over 69 video groups; on validation the video-level AUC was effectively perfect. The corresponding group-level confusion matrix in Fig. 10 shows nearly diagonal behavior, with only two Face2Face groups misassigned. Grad-CAM overlays (Fig. 12) indicate that the model focuses on blending boundaries and central face texture, the regions where manipulation families differ most.

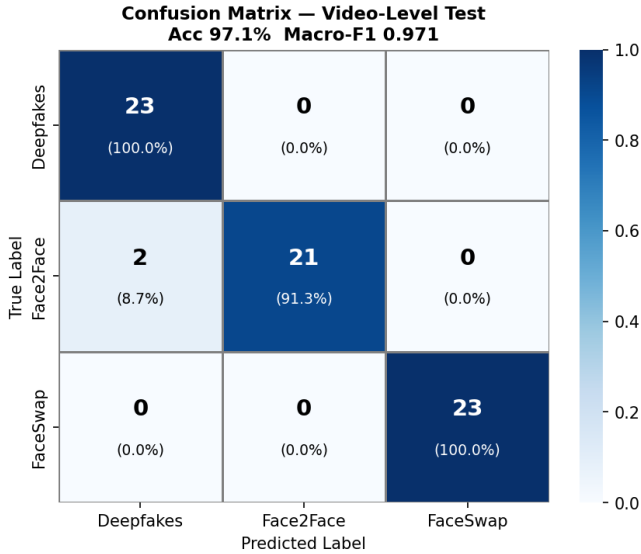


Fig. 10. Manipulation-type video-level (group) confusion matrix on the test set.

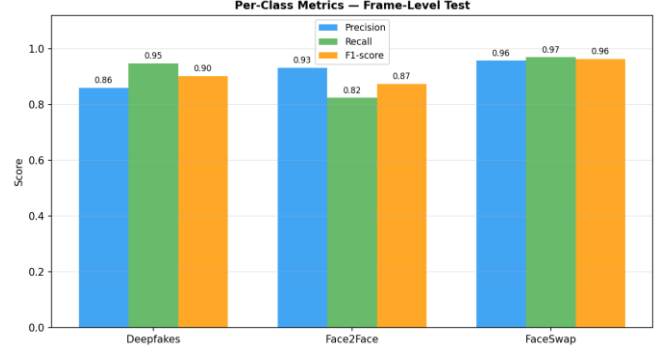


Fig. 11. Per-class precision, recall, and F1 for the manipulation-type classifier.

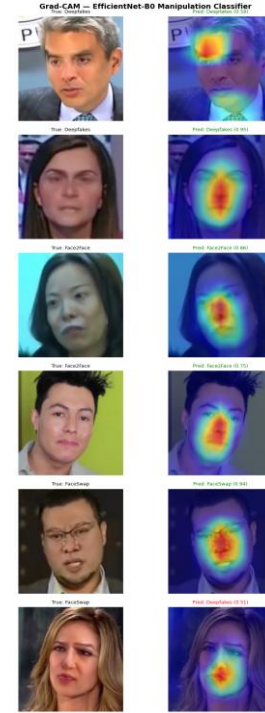


Fig. 12. Grad-CAM overlays for the manipulation-type classifier.

The robustness profile of the classifier (Fig. 13 and Table VII) is notably stronger than the binary detector for mild degradation: at JPEG quality 80 accuracy stays near 89%, and it degrades gracefully down to quality 20. Gaussian noise is far more damaging, with accuracy and Macro-F1 collapsing once the standard deviation exceeds about 5, because additive noise destroys the high frequency cues the frequency branch relies on.

Robustness — EfficientNet-B0 3-Class Manipulation Classifier

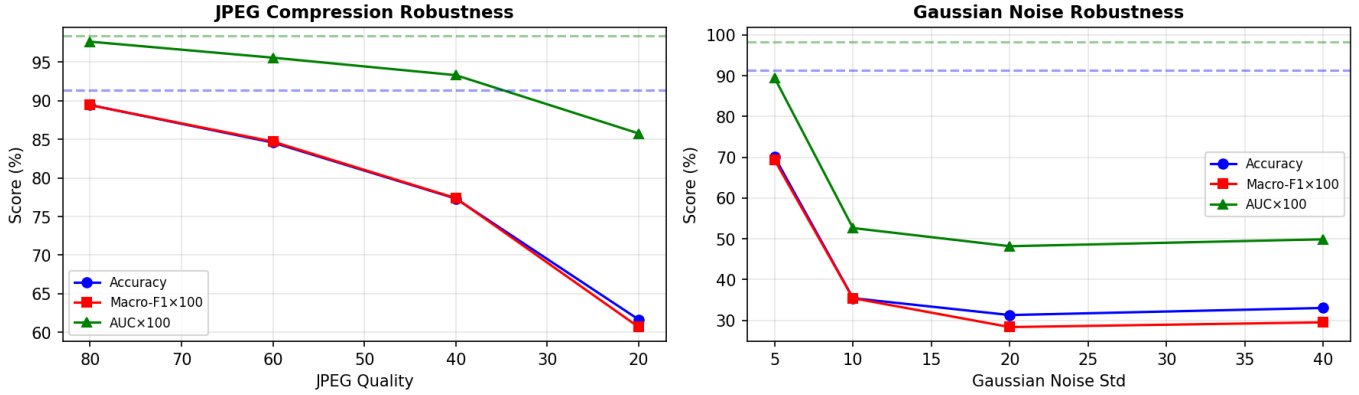


Fig. 13. Manipulation-type classifier robustness under JPEG recompression (left) and Gaussian noise (right).

TABLE VII
MANIPULATION-TYPE ROBUSTNESS (FRAME LEVEL)

Perturbation	Param	Acc.	Macro-F1	AUC
None (baseline)	—	91.38%	0.913	0.983
JPEG	q=80	89.42%	0.894	0.976
JPEG	q=60	84.57%	0.847	0.955
JPEG	q=40	77.32%	0.774	0.933
JPEG	q=20	61.67%	0.607	0.857
Gaussian noise	$\sigma=5$	70.07%	0.693	0.895
Gaussian noise	$\sigma=10$	35.43%	0.354	0.526
Gaussian noise	$\sigma=20$	31.30%	0.284	0.482
Gaussian noise	$\sigma=40$	33.04%	0.295	0.499

D. Unified Five-Class Model (Practical Extension)

The two models above split the task into detect-then-type and deliberately exclude NeuralTextures. As a practical extension we trained a single model that performs both jobs at once, classifying a face directly into five classes: Real, Deepfakes, Face2Face, FaceSwap, and NeuralTextures. It uses a MobileNetV2 backbone with a three-stage schedule (head, partial, then full fine-tuning), MixUp, and test-time augmentation (TTA) that averages predictions over seven augmented views. This is the hardest setting we attempt, because the model must simultaneously reject reality and separate four visually similar forgeries, including NeuralTextures, whose subtle texture edits are the most easily confused.

With TTA the unified model reaches 78.9% accuracy, 0.792 Macro-F1, and 0.965 one-vs-rest Macro-AUC on the test set. A Top-2 accuracy of 93.33% shows the correct label is almost always among the model's top two guesses, which is valuable for a triage tool. Per-class results (Table VIII, Fig. 15) confirm the expected difficulty ordering: FaceSwap is the cleanest class (F1 0.90) and Real is recovered with the highest recall (0.90), while Deepfakes and NeuralTextures are the weakest (F1 0.74 and 0.72), each frequently mistaken for Real exactly the subtle families the three-class model had omitted.

TABLE VIII
UNIFIED FIVE-CLASS MODEL - TEST RESULTS (WITH TTA)

Class	Prec.	Recall	F1
Real	0.68	0.90	0.77
Deepfakes	0.85	0.65	0.74
Face2Face	0.98	0.72	0.83
FaceSwap	0.90	0.90	0.90
NeuralTextures	0.77	0.67	0.72
Macro avg	0.84	0.77	0.79

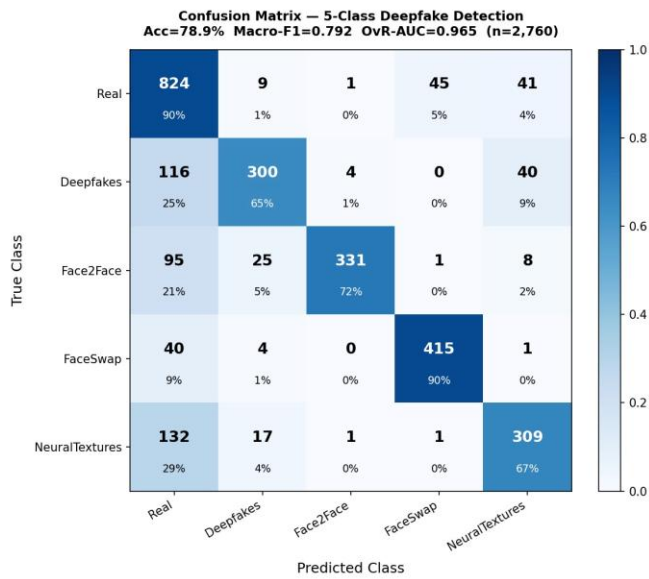


Fig. 14. Five-class confusion matrix on the test set (with TTA). Real is recovered well; NeuralTextures is the most confused class.



Fig. 15. Five-class per-class precision, recall, and F1 on the test set.

Grad-CAM on the unified model (Fig. 18) again localizes to facial regions and blending boundaries, and the error montage

(Fig. 16) shows that most mistakes involve NeuralTextures and Real being confused with Deepfakes, the subtlest manipulations. The per-class confidence distributions (Fig. 19) confirm the model is reasonably calibrated: correct predictions concentrate at high softmax confidence while errors sit lower, and the well-separated classes (FaceSwap, mean 0.78) are more confident than the hard ones (Deepfakes, 0.66; NeuralTextures, 0.67). The model therefore offers a single deployable classifier covering all four FaceForensics++ manipulations plus real, at the cost of lower accuracy than the specialized three-class model, a reasonable trade for completeness. Its three-stage training is shown in Fig. 17.

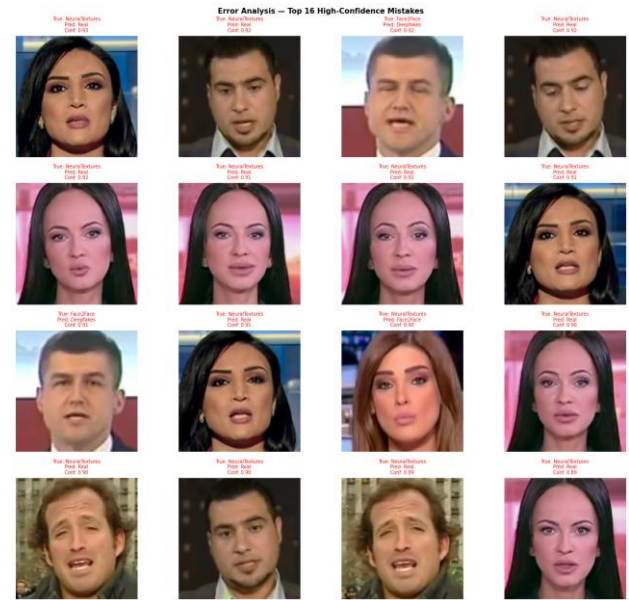


Fig. 16. Five-class model - representative high-confidence misclassifications.

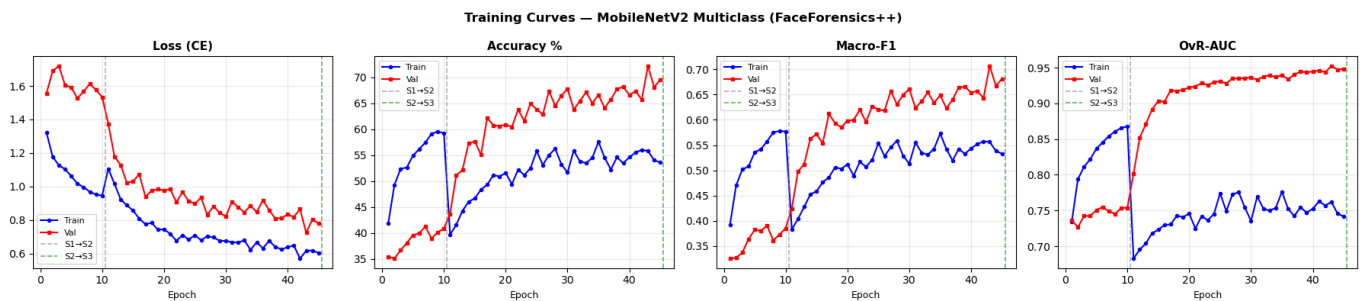


Fig. 17. Five-class model training and validation curves across the three fine-tuning stages.

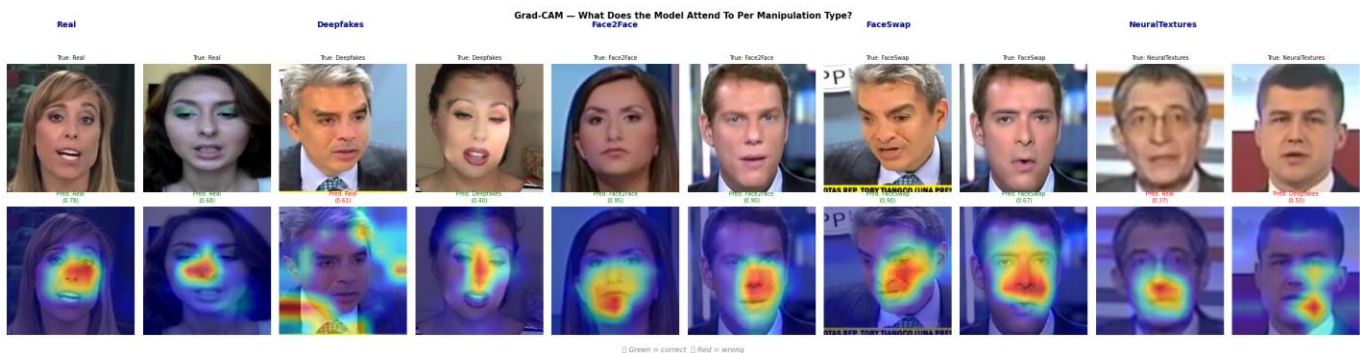


Fig. 18. Five-class model Grad-CAM overlays; attention concentrates on central facial regions for each class.

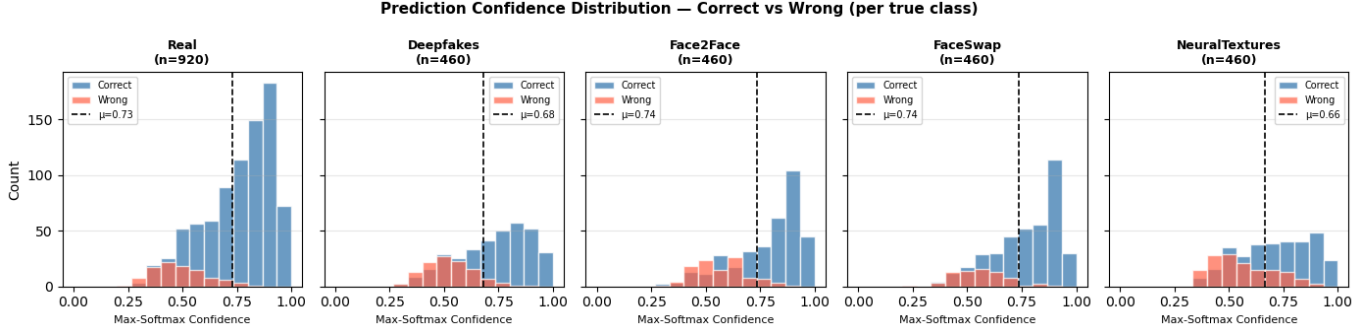


Fig. 19. Five-class prediction-confidence distributions, correct vs. wrong, per true class. Correct predictions cluster at high confidence; most errors fall below the per-class mean (dashed).

E. Error Analysis

To see what each model gets wrong rather than only how often, we inspected the most confident mistakes. For the binary detector (Fig. 20), the dominant failure is real faces under unusual conditions strong off axis pose, heavy compression blur, or partial occlusion being pushed over the threshold into the fake class, together with a smaller set of very clean fakes that slip through as real. This matches the confusion matrix: the minority real class is simply harder, and borderline frames are where the tuned threshold has the least margin.

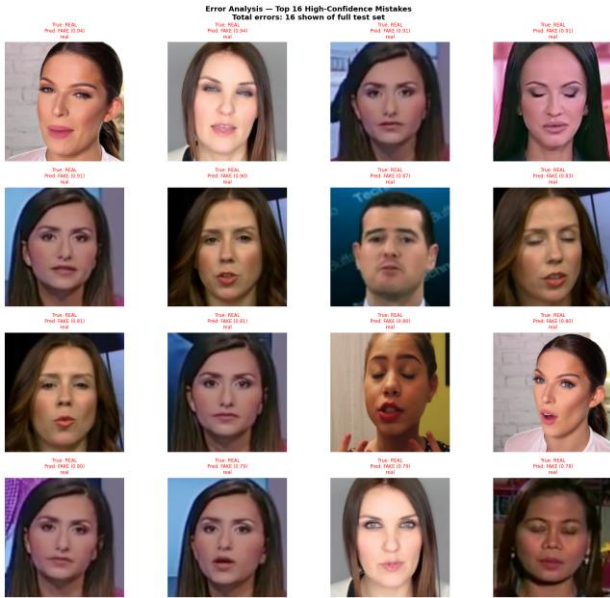


Fig. 20. Binary detector - representative misclassified frames with predicted and true labels.

For the manipulation classifier (Fig. 21), errors cluster almost entirely on the Face2Face–Deepfakes boundary, exactly the pair that shares learned facial-reenactment artifacts; FaceSwap, whose graphics-pipeline seams are visually distinct, is rarely confused. Several of the misread frames are low-detail or motion-blurred, where the frequency fingerprint the model relies on is weakest. Encouragingly, almost none of these frame-level slips survive video-level aggregation, which is why the group accuracy reaches 97%.



Fig. 21. Manipulation classifier - representative misclassified frames; most confusions lie on the Face2Face–Deepfakes boundary.

VI. DISCUSSION

Three findings stand out. First, video-level aggregation consistently outperforms frame-level evaluation for both tasks, raising binary AUC from 0.812 to 0.899 and manipulation accuracy from 91.4% to 97.1%. Because FaceForensics++ is inherently video-based, averaging frame evidence suppresses isolated hard frames and yields the most reliable estimate of real-world performance; it is also the setting that matches deployment, where a verdict is issued per clip.

Second, the two tasks differ sharply in difficulty. Manipulation-type classification is comparatively easy once the frequency branch is added, because each generator leaves a distinctive spectral signature. Binary real-versus-fake detection is harder: the model must reject a diverse and imbalanced real class, and as the error analysis showed, its mistakes concentrate on atypical real frames rather than on any single manipulation. Class weighting and threshold tuning reduce but do not remove this bias, and the ablation makes clear that most of the binary headroom we recovered came from scaling data and deepening the head rather than from the ensemble, which contributed only a final small margin.

Third, the robustness experiments expose a shared weakness. Under heavy JPEG compression or Gaussian noise, Macro-F1 and AUC fall even when raw accuracy looks stable, because

both models retreat toward the majority prediction, a reminder that accuracy alone is misleading under corruption and that Macro-F1 and AUC must be read alongside it. The manipulation classifier is the more noise-sensitive of the two precisely because additive noise corrupts the high-frequency information its frequency branch depends on, yet it tolerates moderate JPEG compression well, which is encouraging given that typical shared media is only lightly compressed.

Beyond benchmark numbers, the system supports single-image inference given one face crop, the binary detector returns a calibrated real/fake probability at the tuned threshold, and the manipulation classifier returns a softmax distribution over the three families (for example, a held-out frame was typed as DeepFakes with about 72% confidence). This confirms a working, lightweight inference path, though we stress that all headline metrics come strictly from the official FaceForensics++ protocol, not from cherry-picked single images.

Deepfake detection also carries ethical weight. A detector that is wrong in the real-as-fake direction can wrongly discredit authentic media, while a missed fake can let a forgery spread; our deliberate handling of the real class and our reporting of both error directions are part of using such a tool responsibly. We also note that a detector trained on a fixed set of manipulation methods may not generalize to unseen generators, so claims of reliability should always be scoped to the families and conditions tested.

Limitations remain. All three models are trained on roughly 300 videos per class rather than the full benchmark, so our numbers are indicative rather than state-of-the-art, and all evaluation is on pre-extracted face crops rather than end-to-end video. The unified five-class model now covers NeuralTextures, but at lower accuracy than the specialized detectors, and like them it remains fragile under heavy noise. Future work includes scaling to the full dataset, adding corruption-aware augmentation to harden all models against noise, analyzing threshold sensitivity under degradation, and integrating face extraction into a single pipeline that runs from raw video to a detection-then-typing decision.

VII. CONCLUSION

We presented a compact, CPU-only system for FaceForensics++ that both detects manipulated faces and identifies the manipulation family. A two-stage MobileNetV2 detector, combined as a simple ensemble with class-weighted training and threshold calibration, reaches 74.06% frame accuracy and 0.899 video-level AUC for binary detection, while an EfficientNet-B0 model fused with an FFT-based frequency branch and trained with MixUp reaches 91.38% frame and 97.10% video-level accuracy for three-class manipulation typing. A unified five-class extension with test-time augmentation reaches 78.9% accuracy while covering all four manipulations plus real in one model. An ablation isolates the contribution of architecture, data scale, and ensembling, and a robustness study with error analysis makes the failure modes explicit. Beyond raw accuracy, the central outcome is a correct evaluation protocol leakage-free pair-based splits, calibrated

thresholds, and both frame- and video-level reporting. These ingredients make the results academically trustworthy and provide a clear foundation for further refinement.

VIII. TEAM CONTRIBUTIONS

Abdelrahman Ibrahim Kamal Eldin: built the CPU training pipeline; integrated the official pair-based FaceForensics++ split; implemented, trained, and tuned the binary MobileNetV2 models and the ensemble; and set up the manipulation-classifier training infrastructure.

Aya Said Abdallah Noah: led result analysis and robustness interpretation; produced the figures and visualizations (training curves, confusion matrices, robustness plots, Grad-CAM); and wrote and assembled the report and presentation materials.

Esraa Mohammed Elsayed: prepared the dataset and preprocessing; verified the split protocol and debugged the identity-leakage issue during frame extraction and index building; and supported the multi-class data pipeline and frequency-branch integration.

Project timeline. The project ran over a four-week development cycle following the graded midterm. Table IX lists each phase with its primary tasks and the responsible member; phases overlapped in practice, and all members reviewed each deliverable before submission.

TABLE IX
PROJECT TIMELINE AND TASK ASSIGNMENT

Phase	Tasks	Lead
Week 1	Scale data to 300 videos; extract 20 frames/video; build official-split index; verify zero leakage	Esraa
Week 2	Train and tune binary MobileNetV2 (two stages); class weights; threshold tuning; v1+v2 ensemble	Abdelrahman
Week 3	Build EfficientNet-B0 + FFT frequency branch; MixUp; train and evaluate 3-class model	Abdelrahman / Esraa
Week 3	Frame- and video-level evaluation; JPEG and Gaussian robustness; Grad-CAM and error analysis	Aya
Week 4	Unified five-class MobileNetV2 (3-stage + MixUp + TTA); evaluation and figures	Abdelrahman / Aya
Week 4	Baselines, ablation, figures, IEEE report writing, and presentation assembly	Aya (all review)

IX. USE OF GENERATIVE AI TOOLS

In line with course policy, the team discloses the use of generative AI tools (ChatGPT, Claude, and GitHub Copilot). These tools were used for debugging assistance after manual attempts, for code review and explanation of concepts (for example, identifying the data-leakage split bug and the missing class weights), and for grammar and phrasing in the report. They were not used to generate experimental results or metric values, to write the methodology or literature review end-to-end, or to implement the core model architectures without the team's understanding. All reported numbers come directly from the team's own training and evaluation runs.

REFERENCES

- [1] A. Rössler, D. Cozzolino, L. Verdoliva, C. Riess, J. Thies, and M. Nießner, “FaceForensics++: Learning to Detect Manipulated Facial Images,” in Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV), 2019.
- [2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2018.
- [3] F. Chollet, “Xception: Deep Learning with Depthwise Separable Convolutions,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2017.
- [4] L. Li, J. Bao, T. Zhang, H. Yang, D. Chen, F. Wen, and B. Guo, “Face X-Ray for More General Face Forgery Detection,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2020.
- [5] H. Qi, Q. Guo, F. Juefei-Xu, X. Xie, L. Ma, W. Feng, Y. Liu, and J. Zhao, “DeepFake Detection by Analyzing Convolutional Traces,” IEEE Trans. Image Process., vol. 29, pp. 10832–10845, 2020.
- [6] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in Proc. Int. Conf. Machine Learning (ICML), 2019.
- [7] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, “mixup: Beyond Empirical Risk Minimization,” in Proc. Int. Conf. Learning Representations (ICLR), 2018.
- [8] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization,” in Proc. IEEE Int. Conf. Computer Vision (ICCV), 2017.
- [9] B. Dolhansky, J. Bitton, B. Pflaum, J. Lu, R. Howes, M. Wang, and C. C. Ferrer, “The DeepFake Detection Challenge (DFDC) Dataset,” arXiv:2006.07397, 2020.
- [10] L. Li, J. Bao, H. Yang, D. Chen, and F. Wen, “FaceShifter: Towards High Fidelity and Occlusion Aware Face Swapping,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2020.
- [11] Y. Qian, G. Yin, L. Sheng, Z. Chen, and J. Shao, “Thinking in Frequency: Face Forgery Detection by Mining Frequency-Aware Clues (F3-Net),” in Proc. European Conf. Computer Vision (ECCV), 2020.
- [12] H. Zhao, W. Zhou, D. Chen, T. Wei, W. Zhang, and N. Yu, “Multi-Attentional Deepfake Detection,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2021.
- [13] A. Eltaweel, “Deepfake Detection on FaceForensics++,” **GitHub repository**, 2026. **[Online]. Available:** <https://github.com/ABDO176ELTAWHEEL/deepfake-detection.git> **[Accessed: Jun. 12, 2026].**

X. APPENDIX: KEY HYPERPARAMETERS

TABLE X
TRAINING CONFIGURATION FOR BOTH MODELS

Setting	Binary	Multiclass
Backbone	MobileNetV2	EfficientNet-B0
Optimizer	AdamW	AdamW
Stage-1 LR	1e-3	3e-4 cos
Stage-2 LR	1e-5	3e-5 cos-WR
Batch size	16	16
Label smooth.	0.05	0.10
Unfrozen (S2)	Last 4	Last 3
Imbalance	Sampler + wts	MixUp $\alpha=0.3$
Early stop	Patience 4	Patience 6
Frames/video	20	20

TABLE XI
MANIPULATION PER-CLASS RESULTS (GROUP LEVEL)

Class	Prec.	Recall	F1
Deepfakes	0.92	1.00	0.96
Face2Face	1.00	0.91	0.95
FaceSwap	1.00	1.00	1.00

Class	Prec.	Recall	F1
Macro avg	0.97	0.97	0.97